

Class work: graphing one numerical variable

Daniel Coblentz

2-4-2024

Getting set up!

Task 1: Edit the header to add your name.

As always, we start by loading some R packages.

Task 2:

Today, in addition to the `tidyverse` package, we will also use the `ggdist` and `ggribes` packages to help us graph our data. I have written the code below to load the `tidyverse` package. Fill in the appropriate code to load the other two packages we will need today.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggdist)
```

```
library(ggribes)
```

```
##
```

```
## Attaching package: 'ggribes'
```

```
##
```

```
## The following objects are masked from 'package:ggdist':
```

```
##
```

```
##   scale_point_color_continuous, scale_point_color_discrete,
```

```
##   scale_point_colour_continuous, scale_point_colour_discrete,
```

```
##   scale_point_fill_continuous, scale_point_fill_discrete,
```

```
##   scale_point_size_continuous
```

Loading and looking at some data

Task 3:

First load the data file called `oscars.csv`, which is in the folder called `data` in this project. We will give the new dataframe the name `oscars`. Add the appropriate command on the right side of `<-` to load the data.

In your first r assignment you used a command to read a file from a folder called `data`, so go back to that assignment to look at how we did that.

```
oscars <- read_csv('data/oscars.csv', show_col_types = FALSE)
head(oscars, 5)
```

```
## # A tibble: 5 x 8
##   oscar_no oscar_yr award      name      movie    age birth_pl birth_date
##   <dbl>    <dbl> <chr>      <chr>      <chr>  <dbl> <chr>    <chr>
## 1         1      1929 Best actress Janet Gaynor  7th H~    22 Pennsylv~ 10/6/1906
## 2         2      1930 Best actress Mary Pickford Coque~    37 Canada   1892-04-08
## 3         3      1931 Best actress Norma Shearer The D~    28 Canada   8/10/1902
## 4         4      1932 Best actress Marie Dressler Min a~    63 Canada   1868-11-09
## 5         5      1933 Best actress Helen Hayes   The S~    32 Washing~ 10/10/1900
```

You will find the codebook (our textbook calls this a **data dictionary**) for this data set at <https://www.openintro.org/data/index.php?data=oscars>. The “birth month”, “birth day”, and “birth year” variables are combined here in the single `birth_date` variable. In a new browser tab, go to the codebook and familiarize yourself with the variable descriptions.

Task 4:

Use one of the ways we have learned to view the dataframe to open the data set so you can look at it in detail.

```
head(oscars, 10)
```

```
## # A tibble: 10 x 8
##   oscar_no oscar_yr award      name      movie    age birth_pl birth_date
##   <dbl>    <dbl> <chr>      <chr>      <chr>  <dbl> <chr>    <chr>
## 1         1      1929 Best actress Janet Gaynor  7th ~    22 Pennsylv~ 10/6/1906
## 2         2      1930 Best actress Mary Pickford Coque~    37 Canada   1892-04-08
## 3         3      1931 Best actress Norma Shearer The ~    28 Canada   8/10/1902
## 4         4      1932 Best actress Marie Dressler Min ~    63 Canada   1868-11-09
## 5         5      1933 Best actress Helen Hayes   The ~    32 Washing~ 10/10/1900
## 6         6      1934 Best actress Katharine Hep~ Morn~    26 Connect~ 5/12/1907
## 7         7      1935 Best actress Claudette Col~ It H~    31 France   9/13/1903
## 8         8      1936 Best actress Bette Davis   Dang~    27 Massach~ 4/5/1908
## 9         9      1937 Best actress Luise Rainer   The ~    26 Germany  1/12/1910
## 10        10      1938 Best actress Luise Rainer   The ~    27 Germany  1/12/1910
```

```
n_cases <- nrow(oscars)
n_cases
```

```
## [1] 190
```

Task 5: Answer the following questions.

How many cases are there? How can you tell?

Answer: 182(out dated value), 190 the code book mentions there are 182 observations however using the ‘nrows’ function we can count the number of rows(cases) in the file which comes out to 190.

Are the names of the winners mostly familiar? Are there names you do not recognize?

Answer: No, I do not recognize the names.

Are there any missing entries in the first 25 rows of the data set?

Answer: No

```
# get first 25 rows
first_25_rows <- head(oscars, 25)

# check missing values
any_missing_values <- any(is.na(first_25_rows))

any_missing_values

## [1] FALSE
```

What variables in this dataset are numerical? How can you tell?

Answer: - oscar_no - oscar ceremony number. - oscar_yr - year the Oscar ceremony was held. - age - Age at which the actor or actress won the Oscar.

exp: These variables are numerical because they represent values that can be measured or counted.

Task 6:

Make a table of the Best actor winners ranked in order of age from youngest to oldest. Keep only the variables name, age, and movie. If neither you nor your partner remembers how to do this, look back at your Counties R activity from last week or at the R Cheat-Sheet you started to make today.

```
oscars %>%
  filter(award == "Best actor") %>%
  select (name, age, movie) %>%
  arrange(age)

## # A tibble: 95 x 3
##   name          age movie
##   <chr>         <dbl> <chr>
## 1 Adrien Brody    29 The Pianist
## 2 Marlon Brando   30 On The Waterfront
## 3 Richard Dreyfuss 30 The Goodbye Girl
## 4 Maximilian Schell 31 Judgment at Nuremberg
## 5 Nicolas Cage    31 Leaving Las Vegas
## 6 James Stewart   32 The Philadelphia Story
## 7 Daniel Day-Lewis 32 My Left Foot
## 8 Eddie Redmayne  32 The Theory of Everything
## 9 Clark Gable     33 It Happened One Night
## 10 Charles Laughton 34 The Private Life of Henry VIII
## # i 85 more rows
```

Task 7:

Make a table of the Best actress winners ranked in order of age from oldest to youngest. Keep only the variables name, age, and movie. Again, if you can't remember how to do this, look back at the Counties activity or at the Cheat Sheet you worked on for homework.

```
oscars %>%
  filter(award == "Best actress") %>%
  select (name, age, movie) %>%
  arrange(desc(age))
```

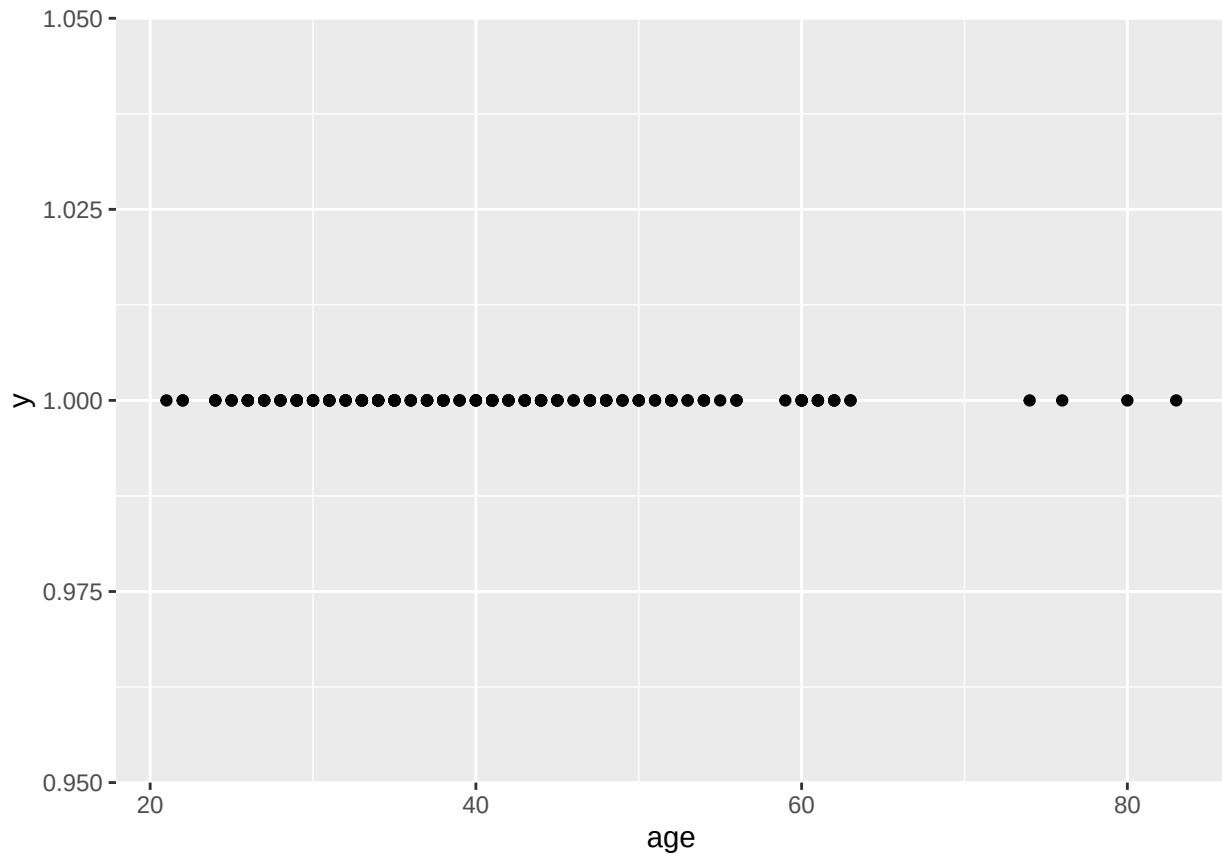
```
## # A tibble: 95 x 3
##   name          age movie
##   <chr>        <dbl> <chr>
## 1 Jessica Tandy      80 Driving Miss Daisy
## 2 Katharine Hepburn   74 On Golden Pond
## 3 Marie Dressler      63 Min and Bill
## 4 Frances McDormand   63 Nomadland
## 5 Meryl Streep       62 The Iron Lady
## 6 Katharine Hepburn   61 The Lion in Winter
## 7 Geraldine Page      61 Trip to Bountiful
## 8 Helen Mirren        61 The Queen
## 9 Katharine Hepburn   60 Guess Who's Coming to Dinner
## 10 Frances McDormand  60 Three Billboards Outside of Ebbing Missouri
## # i 85 more rows
```

Graphing a single numerical variable using dotplots, histograms, and frequency polygons

Dotplots

We can graph the (numerical) `age` variable as a dotplot using the following chunk. The y-axis of the graph will look strange. We could fix this, but we will not worry about it for now. Go ahead and run this code.

```
ggplot(data = oscars,
       mapping = aes(x = age,
                     y = 1)) +
  geom_point()
```



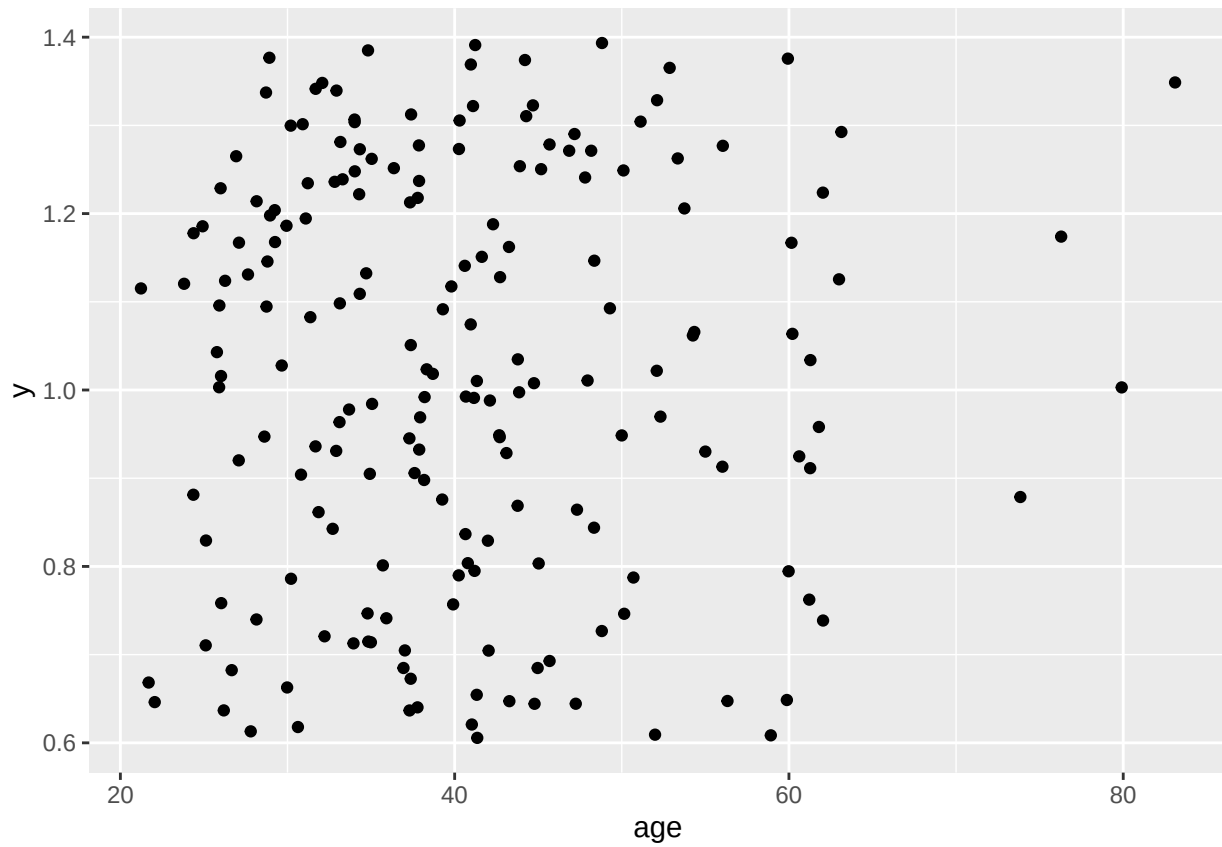
This graph is hard to interpret because of “overplotting,” which is when data points overlap in a visualization, making it difficult to see individual data points. To help with overplotting, we can try two other kinds of graph: a “jittered” dotplot and a stacked dotplot. Below, we will make both of these kinds of graph.

To make the jittered dotplot, use the same code as before but swap the `geom_point()` for a new geom: `geom_jitter()`.

To make a stacked dotplot, the geom you will need is `geom_dots()`.

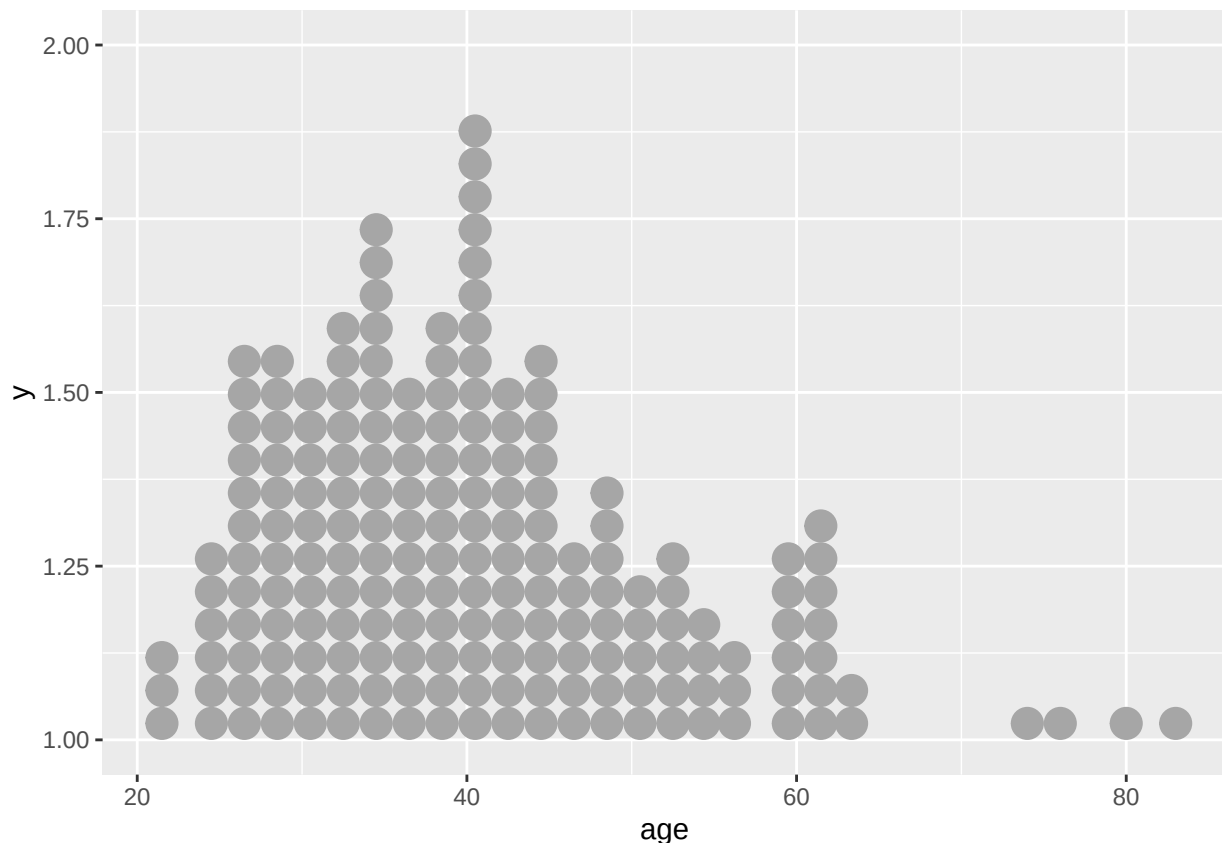
Task 8: Try a jittered dotplot here

```
ggplot(data = oscars,
       mapping = aes(x = age,
                     y = 1)) +
  geom_jitter()
```



Task 9: Try a stacked dotplot here

```
ggplot(data = oscars,  
       mapping = aes(x = age,  
                     y = 1)) +  
  geom_dots()
```



Task 10:

Based on the graphs you created above, write two or three sentences about the distribution of ages among the Oscar winners. From the graph above, we can see that the distribution of ages appears to be concentrated primarily between the ages of 30 and 60. There is a peak around the 40s, suggesting most Oscar winners are in this age range. Then, the frequency of winners decreases sharply for ages below 30 and above 60.

Be specific in your writing, referring to ages of Oscar winners rather than dots on the dotplot. Some things you might write about: Between what values do most Oscar winners' ages lie? What would you say is an (approximate) average age for the Oscar winners? Are there any winners who do not follow the general trend? Highlight these. Throughout, write in complete sentences.

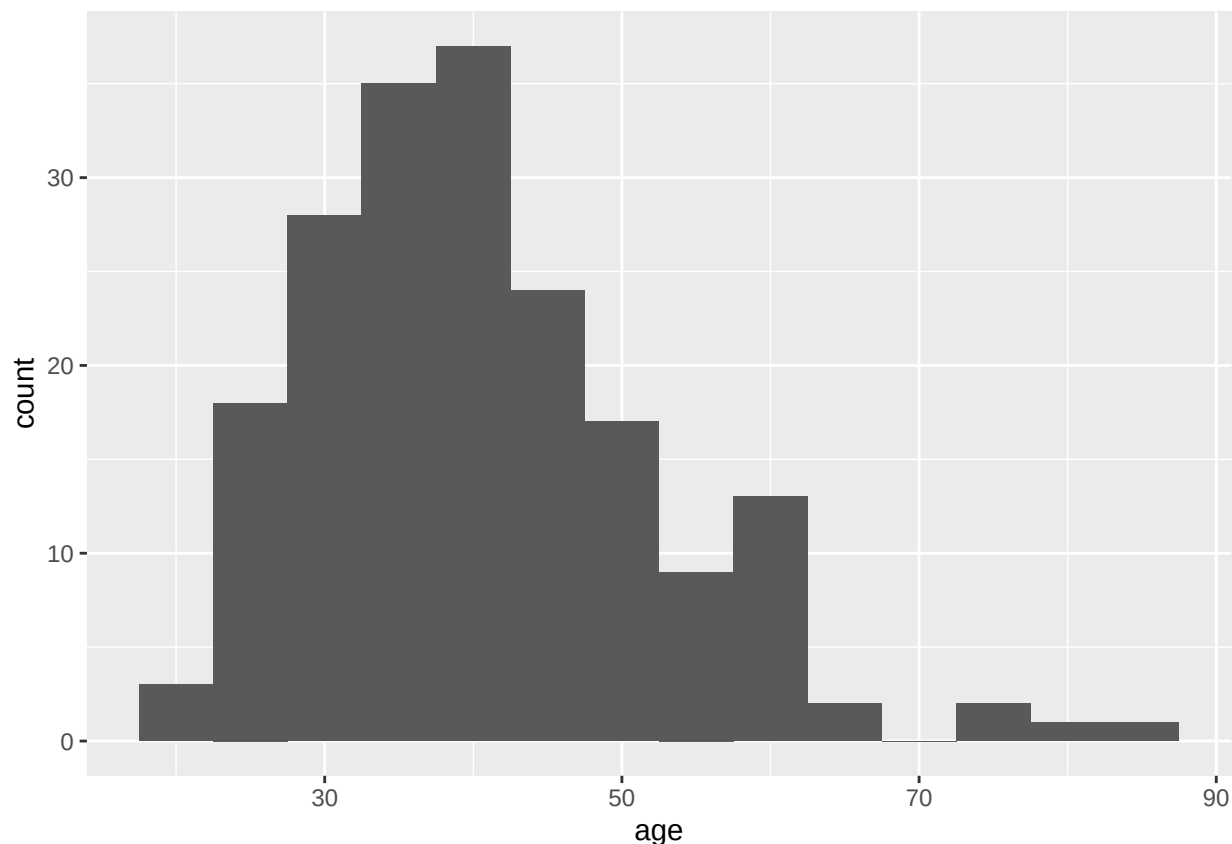
Answer: From the graph, it is clear that most oscar winners are between 30 and 60 years old. The data suggests an average age around 45 years old, as I mentioned earlier, there is a peak here. While the general trend shows fewer winners outside this central age group, there are a few exceptions. For example, a small number of winners are in their 20s and some in their late 70s, breaking away from the general trend.

Histograms

Task 11:

We can make a histogram of the data as well. To do this, copy the code from one of the dotplots above, delete the `y=1` part of the aesthetic. For the geom, use `geom_histogram()`, without the backticks.

```
ggplot(data = oscars,
       mapping = aes(x = age,
                     )) +
  geom_histogram(binwidth = 5)
```



Task 12:

What is the default bin width for the histogram you just created?

Answer: 2

Task 13:

Change the default binwidth to 5 by typing `binwidth = 5` inside the parentheses for the geom in your code above. Run the code. Does it work?

Answer: yes!

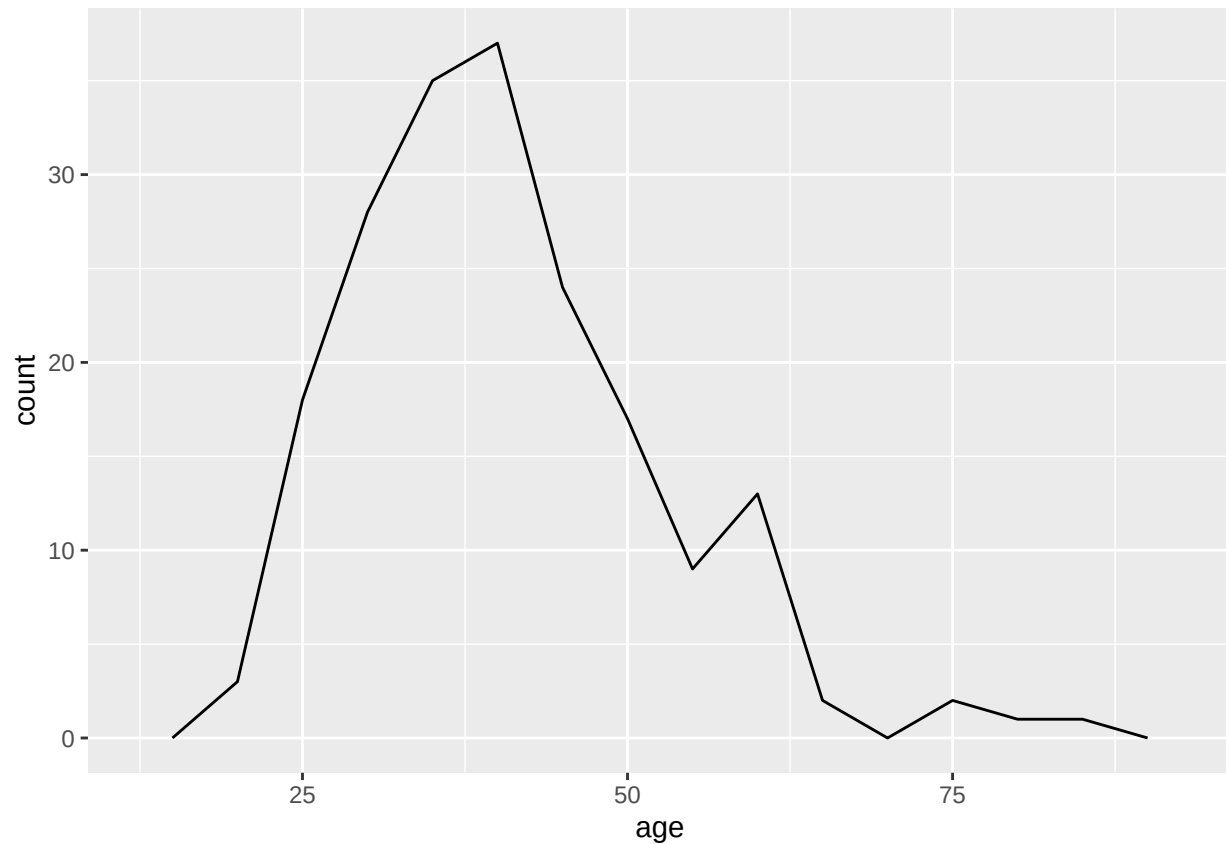
Frequency polygons

Another type of graph that is very similar to a histogram is a frequency polygon. These graphs are especially useful for comparing distributions in two groups. The code is almost the same as for histograms. The only thing that changes is the geom; for that you should use `geom_freqpoly`, again without the backticks. Modify your code to use bins of 5-year age groups.

Task 14:

Follow the instructions above for making a frequency polygon for the `age` variable using bins of 5-year age groups.


```
ggplot(data = oscars,
       mapping = aes(x = age)) +
  geom_freqpoly(binwidth = 5)
```



Extra problems from class (2/7/2025): # quantile ranges:

```
oscars %>%
  summarize(q025 = quantile(age, 0.025),
            q25 = quantile(age, 0.25),
            q75 = quantile(age, 0.75),
            q975 = quantile(age, 0.975))
```

```
## # A tibble: 1 x 4
##   q025  q25  q75  q975
##   <dbl> <dbl> <dbl> <dbl>
## 1    24    32    47    63
```

find the mean:

```
oscars %>%
  summarize(mean_age = mean(age, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   mean_age
##   <dbl>
## 1    40.6
```

median:

```
oscars %>%  
  summarize(median_age = median(age, na.rm = TRUE))
```

```
## # A tibble: 1 x 1  
##   median_age  
##       <dbl>  
## 1         39
```

std.dev:

```
oscars %>%  
  summarize(sd_age = sd(age, na.rm = TRUE))
```

```
## # A tibble: 1 x 1  
##   sd_age  
##     <dbl>  
## 1    11.6
```

data range

```
oscars %>%  
  summarize(max(age), min(age))
```

```
## # A tibble: 1 x 2  
##   `max(age)` `min(age)`  
##       <dbl>    <dbl>  
## 1         83        21
```

solutions

mean: 40.6 median: 39 std.dev:11.6 data goes from: 21 - 83 q025 = 24 q025 = 32 q75 = 47 q975 = 63